

CSC 510 - Project 1b Report

Ethan Goodman
NC State University
Raleigh, North Carolina, US

Morgan Simon
NC State University
Raleigh, North Carolina, US

Moe Ko
NC State University
Raleigh, North Carolina, US

Savanah Hendzel
NC State University
Raleigh, North Carolina, US

1 Market Survey

There are currently two official websites from NC State that provide similar menu data to what is proposed by PackPlate. The NC State Dining website supplies the hours of every campus dining option and a short description of each while NetNutrition lists the menus and nutrition facts about what each dining hall serves on a daily basis. The NC State App Development Club also offers a mobile version called FoodFinder that combines the two websites for convenience.

However, all of these options only provide basic information about the hours and the food options available. While there is basic filtering for dietary restrictions, further nutritional information is not provided.

2 Mission Statement

PackPlate is a website that helps NC State students decide where to eat on campus. It collects dining hours, menus, and nutrition information from every campus dining location into one place, and lets students filter the menus by their dietary restrictions and goals, such as allergies, vegetarian, halal, or a protein target. Students can enter constraints for several people at once, so a group with different diets can find a location where everyone has something to eat. Students can also report how busy each location is, so others know what to expect before walking over. The information comes from what NC State Dining publishes, and the site shows when that information is incomplete rather than filling in gaps. It is aimed at students on meal plans, especially those living on campus who eat most of their meals in the dining halls.

Answering "what can I eat right now, given my dietary restrictions" currently requires roughly 20 navigations across two separate websites. Hours are published on the NC State Dining site; menus are published in a separate NetNutrition portal, which carries no hours information. A student must therefore consult one site for hours, then repeat a select-location, select-date, select-meal sequence in the other for each of the five dining halls. Because the menu portal has no concept of the current time, it presents full menus for locations that have already closed, so a student can read and act on a menu for a hall that is no longer serving. PackPlate answers the same question in a single view, showing only locations open at the time of the request

2.1 Stakeholders & Why they Matter

- **NC State Students** - Need fast, accurate information when choosing where to eat.

- **Students with allergies** - Could be harmed by incomplete or incorrect allergen information.
- **Students with religious or dietary restrictions** - Need information about halal, vegetarian, vegan, and other requirements.
- **Groups of friends** - Need to find one location that works for several people simultaneously.
- **NC State Dining** - Owns or publishes the underlying menu, nutrition, and hours data.
- **Dining-hall staff** - May receive questions or complaints when published information is incomplete.
- **NC State administrators** - Care about student services, accessibility, privacy, and institutional reputation.
- **PackPlate maintainers** - Must monitor stale data, scraper failures, crowd reports, and deployment issues.
- **University legal/privacy staff** - Need the product to avoid unsafe claims and unnecessary storage of dietary information.

3 Milestones

3.1 Before

- Audited an existing full-stack food application that the team did not write.
- Produced 20 evidence-backed use cases from the existing system.
- Ran identical prompts across 3 cloud LLMS and 1 locally hosted LLM
- Identified defects, including an allergen check that was not enforced on the server.
- Tested model reliability by recording incorrect, partial, and hallucinated answers.

3.2 Now

- Build a responsive PackPlate prototype showing NC State dining locations, hours, menus, and nutrition data.
- Implement dietary filtering with fits, conflicts, and data incomplete results.
- Add multi-person filtering so a group can compare several dietary profiles against the same locations.
- Add simple student crowd reports using a small set of choices such as not busy, moderate, and crowded.
- Test the filtering logic against a hand-labeled menu dataset and evaluate whether student groups can choose a location in under two minutes.

3.3 After

- Use class schedules and walking distance to suggest dining locations during schedule gaps.
- Add coordination features for groups, such as shared meal links or voting.
- Use accumulated reports to estimate busy periods by location and time.
- Improve ingestion reliability and support additional data sources or campus locations.
- Evaluate whether a native mobile application is justified after testing the web prototype.

4 Architecture

4.1 Architecture Pattern

PackPlate will be built as a layered monolith with a client-server structure. Meaning a single web front end will talk to a single back end application. That backend application will then be split into three layers: The API routes that receive requests, the business logic that runs the dietary filtering and combines crowd reports, and the data access layer that reads and write to the database. Also, the dining data itself will not be manually entered into the database. Instead of this, a separate scheduled job will scrape what NC State Dining publishes, then parse, clean, and label it before writing into the database. This means a problem with scraping will only stay inside of that job instead of spreading into the rest of the site. This is the chosen architecture because it works best for the constraints of the project.

Our team consists of four people, each working about ten hours per week on this project for about one month. And the monolith architecture allows for a single application that can be run, tested, and deployed by everyone, meaning it allows for an easier and faster build, deployment and debugging time than if it was a set of separate services. Furthermore, this design also lets us divide work, since one person can build the filtering while another can do something else, like build the scraping, without the two getting in each others way. With this, we do acknowledge there are tradeoffs with this design. For example, the monolith architecture cannot scale well or deploy its parts separately. The scraper and the part of the site that answers a student request grow and ship as one unit, and if one of them ever needs to scale on its own we will have to split the scraping job out into its own service later. However, we accept those limitations in exchange for a system we can finish and debug this month. Figure 1 shows our system design.

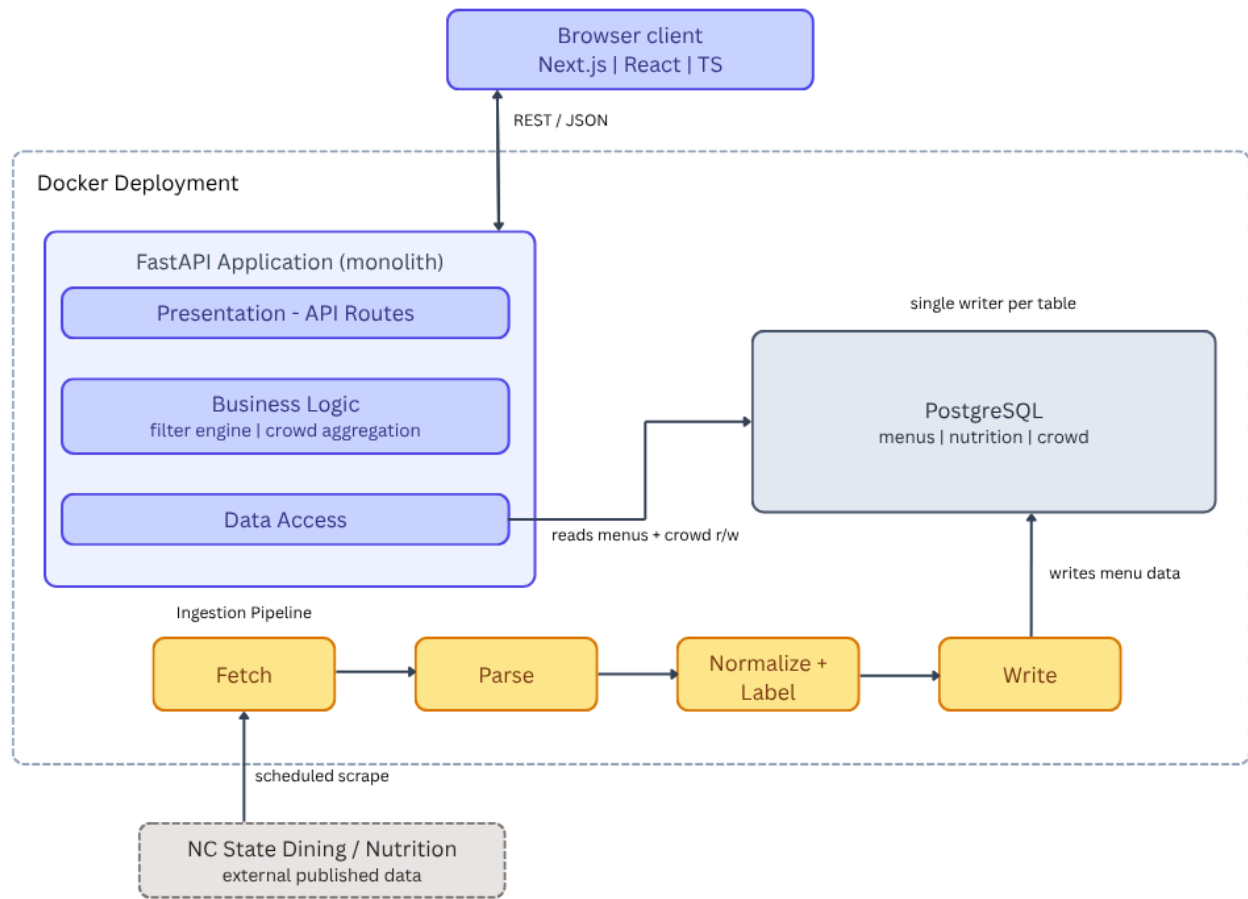


Figure 1: PackPlate architecture: a layered client-server monolith, with a pipe-filter ingestion pipeline. The API and the ingestion job share one database but each owns its own tables (single writer per table).

5 Prompt Report

5.1 Approach

We tested three cloud models, Claude Opus 5, ChatGPT Sol 5.6 and Gemini 3.5 Flash, and one local model, Qwen2.5-Coder-14B, by running the same six prompts on each and recording a verdict for every response.

Our prompt set derives from the set of twelve starter prompts. We wrote six prompts of our own instead of running those twelve directly. Some combine two starters that ask related questions, and all six are enriched with details specific to PackPlate. Every prompt also states our actual budget, four students at roughly ten hours a week for one month, so that nothing came back designed for a longer timeline.

- **P1, rivals and feature matrix.** Asks for the ten closest competing products, then scores each against the four things PackPlate claims to do. The second table is ours, and it is what turns a list of competitors into an answer about where the gap is.
- **P2, the gap with receipts.** States our claimed gap and asks the model to attack it. We supplied the evidence we had gathered and also a list of what we had looked for and could not find.
- **P3, mission, metric and stakeholders.** Asks for three candidate mission statements and a table of everyone affected. We added the requirement that each mission contain a measurable claim with a metric, a baseline and a threshold.
- **P4, milestones and data dependency.** Grades our draft milestones against the hours we have, and checks that each one has a test rather than just a build. We added a data-risk section, because PackPlate depends on one source we do not control.
- **P5, support material.** Asks what laws, standards and licences would change our design. We narrowed it to allergen labelling, the terms of use of the pages we read, and accessibility.
- **P6, red team and allergen liability.** Asks the model to attack the proposal as hard as it honestly can. We added a second role, a lawyer acting for the parent of a student with a severe nut allergy, because the worst thing PackPlate can do is call a dish safe when it is not.

P2 and P4 were the most useful. P2 asked the models to disprove our own claim, and three of the four argued that it does not hold up. P4 asked what could go wrong with the data source the project depends on, which none of the starter prompts cover. P5 was the least useful, because the models returned long lists of only loosely related sources.

The two prompts that had to run on every model are the market survey and the adversarial one. P1 is our market survey and P6 is our adversarial prompt, and both ran on all four models.

The three cloud models can search the web. The local model cannot, so anything it says about the market is recalled rather than checked. That difference explains the most important disagreement in the run.

Every prompt, response, verdict and note is committed at `proj_1b/prompts/all_models.csv`.

We also ran the LLM-as-a-critic prompt after we decided on our design. The prompt was as follows: "You are a reviewer who has killed three projects like this. Here is our box diagram and our four milestones. For each arrow: what breaks under load, what breaks in week 3, which module will the merge conflicts live in? Cite the box, or write UNKNOWN. Keep what survives your own check; the critique that names a real seam goes in the report." The full results are in `proj_1b/prompts/architecture_critique.txt`.

The LLM found several missing pieces in the diagram that would support the system operationally. It critiqued the exclusion of low-level tools and noted that the FastAPI monolith and uncached PostgreSQL reads would likely become bottlenecks as activity increased. By week 3, the most likely problems would be frontend/backend field-name mismatches, the upstream scraper breaking, and the crowd level counts being incorrect due to concurrent writes. The diagram did hold up to scrutiny when it came to structural logic for the business layer.

5.2 Caught errors

Table 1 gives the verdict for each prompt on each model. Every response was read against its sources. What follows is what we found wrong, and how.

- **Qwen listed a product that does not exist.** It gave "Dine360" as a nutrition tracker at `dine360.com`. We searched: DINE360 is a beverage company's research study on foodservice purchasing. It also gave "Yumzie" as a restaurant finder; a product by that name exists but is a cooking assistant at a different address.
- **Qwen broke an explicit rule in the same response.** P1 states that products it cannot confirm should be left out. Four were included anyway, each marked "Unconfirmed URL."
- **Every model invented a baseline.** P3 asks for a measurable claim with a metric, a baseline and a threshold, and gives an example with the numbers left as placeholders. We supplied no figure of our own because we had not measured one. All four filled the placeholder in regardless: Qwen 30 minutes, Gemini 15, ChatGPT 8, Claude 6. Nothing measured any of them, and the fivefold spread across the same task shows a model will fill an empty metric slot rather than leave it empty. Only Claude flagged its own figure as an estimate. The fault is ours as much as the models': the prompt should have said to write "to be measured" rather than a number.
- **Qwen would not argue against the project.** P6 asks the model to attack the proposal as hard as it honestly can. Qwen answered in 6.5 seconds, the fastest response in the run, saying it did not have the ability to criticise the project and offering to help with something else instead. All three cloud models answered the same prompt without hesitation.

5.3 The market survey and the two-model rule

A competing product counts only if two models name it, or one gives a live URL.

Named by two or more models. The NC State Dining site and its Menu and Nutrition Portal, which runs on CBORD NetNutrition; OnCampus; Nutrislice; Dine On Campus; Waitz; and Grubhub campus dining. UPlate was also named twice, by Claude and ChatGPT, but in the gap prompt rather than the survey. Claude and Gemini disagree on who operates Dine On Campus, one naming Sodexo and the other Chartwells, and we did not resolve it.

Named once. Google Maps, SupperSquad, Picknic and My-FitnessPal from Claude; CampusFuel, Plateform, Pixx and CampusWaits from ChatGPT; CollegeMacros, DormDish AI and Swipe Dining from Gemini. Each came with a URL, which is what the rule allows a single naming on, but we have not opened them.

Failed the rule, or missed entirely. Every product named only by Qwen failed: Dine360, Yumzie, CampusEats, CampusFoodApp, Campus Life App, WaitTime and Nutrium, four of which Qwen itself marked “Unconfirmed URL.” We searched two — Dine360 does not exist as described, and Yumzie is a different kind of product at a different address. Going the other way, one real competitor was missed by all four models: FoodFinder, an Android app built by the NC State App Development Club that reads the same menu source we plan to read. It is public on GitHub and it is the closest competitor PackPlate has. Three of our four models could search the web and none found it.

5.4 The disagreement on P2

P2 gave each model a claim we had written and asked it to try to disprove it. The claim was that no existing tool lets a group of students each enter their own dietary restrictions and see which campus dining locations work for everyone. We supplied the evidence we had collected, and also a list of things we had looked for and failed to find.

Three of the four models concluded that the claim did not hold up.

Claude found that the software behind NC State’s menus already lets one person filter on several allergens at the same time, which means most of what we proposed to build already exists. ChatGPT agreed that students have a real problem finding dining information, but said we had no evidence that group filtering is the solution they want. Gemini went furthest and said the claim was dead, because nobody has ever asked for the feature we planned to lead with.

Qwen agreed with us. It said the gap was real.

The difference between them is that the first three can search the web and Qwen cannot. Qwen named no competing product anywhere in its answer, and the evidence it offered in support was the evidence we had written into the prompt ourselves, repeated back as though it had found it. At one point it listed the absence of group filtering as a confirmed finding and then marked the same item as not found two lines later.

This is the most useful result in the run. The only model that agreed with us was the one that had no way to check, and the three that could check all disagreed. If we had judged by counting how many models supported the claim, or run only the local model, we would have carried a claim into the report that three better-informed models had already rejected.

5.5 Strengths and weaknesses

Claude Opus 5 was the most reliable across all six prompts, and the only model that pointed out the limits of its own answers without being asked. Its weakness is length. Its answers were far longer than the others’ and included material we had not asked for.

ChatGPT Sol 5.6 was the most careful about what it claimed. It stated a rule at the start of the market survey, that unknown does not mean no, and followed it. It was also the only model that checked the evidence we gave it. Its weakness is that it gave us more than we could use and did not say which parts mattered most.

Gemini 3.5 Flash made the strongest single argument against the project, pointing out that students with mixed diets already solve this problem by defaulting to a large food hall. Its weakness is accuracy. We marked two of its six answers as containing made-up information, more than any other model.

Qwen2.5-Coder-14B cost nothing to run and answered five of six prompts. Its weaknesses are that it cannot check anything it says, it would not argue against the project when asked, and it was the only model to support a claim the other three rejected.

5.6 Conclusion

We compared four models on the same six prompts and recorded a verdict for every answer. The clearest pattern was that the three models able to search the web all disagreed with our central claim, while the one that could not agreed with it. The local model also produced the only invented product in any market survey, and would not carry out the prompt asking it to argue against the project. We report its results rather than leave it out, but we did not rely on them.

Table 1: Verdict for each prompt, by model. Every prompt ran on every model. A verdict of correct means every checkable claim in the response held up when we opened the source ourselves; partial, wrong or hallucinated means at least one did not.

Prompt	Claude Opus 5	ChatGPT Sol 5.6	Gemini 3.5 Flash	Qwen2.5-Coder-14B
P1: Rivals and feature matrix	correct	partial	hallucinated	hallucinated
P2: The gap with receipts	correct	correct	partial	wrong
P3: Mission, metric and stakeholders	correct	partial	partial	partial
P4: Milestones and data dependency	correct	correct	correct	partial
P5: Support material	partial	partial	hallucinated	partial
P6: Red team and allergen liability	correct	correct	correct	wrong

GitHub Repository: <https://github.com/morgansimon425/PackPlate>